Rekomendacje oparte o embeddingi

Systemy Rekomendacyjne 2026/2027

Charakterystyka problemu

- Nowe elementy do zarekomendowania pojawiają się często
  - a w przypadku np. portali informacyjnych, żyją krótko (zazwyczaj max. 100 godzin)
- System powinien działać niemal w czasie rzeczywistym i reagować na zmienne trendy zainteresowań użytkowników
- Ważna jest nie tylko aktywność użytkowników, ale także treści, które rekomendujemy

Dane

- Historia użytkowników
- Strumień zdarzeń od użytkowników
- Teksty (lub obrazy, audio, wideo, …)

Metody oparte o algorytmy bandyty

Po prostu bandyta

- Każdy artykuł traktujemy jako ramię bandyty
- Na podstawie zebranych zdarzeń aktualizujemy payout każdego ramienia
  - Najlepiej sprawdza się optymalizacja znormalizowanych metryk, np. CTR

Po prostu bandyta

- Zalety:
  - Bardzo prosta implementacja, zarówno samego algorytmu jak i infrastruktury gromadzącej dane
  - Minimalny stopień skomplikowania z punktu widzenia aplikacji frontendowej
- Wady:
  - Nie uwzględniamy zmienności zainteresowań użytkowników w czasie (ograniczony wpływ, ponieważ czas życia artykułów jest krótki)
  - Nie uwzględniamy zainteresowań użytkowników (każdemu rekomendujemy te same treści)
  - Zagładzamy większość artykułów, prezentując tylko te najbardziej interesujące dla ogółu (szczególnie dotkliwe w przypadku dużej puli różnorodnych artykułów)

Bandyta z wygasającą wiedzą

- Zamiast przechować dane o payoucie materiału z całego okresu jego istnienia (np. średni CTR od momentu opublikowania artykułu), implementujemy mechanizm wygasania wiedzy z czasem
- Przykłady:
  - Okno czasowe, czyli przechowywanie danych z N ostatnich godzin
  - Wykładnicze wygasanie, czyli regularne (np. co kilka minut) przemnażanie przechowywanych wartości przez $\alpha < 1$

Bandyta z wygasającą wiedzą

- Zalety:
  - Stosunkowo prosta implementacja (choć implementacja podejścia z oknem czasowym wymaga wykorzystania odpowiednich narzędzi, np. *Apache Druid*)
  - Minimalny stopień skomplikowania z punktu widzenia aplikacji frontendowej
  - Uwzględniamy zmienność zainteresowań użytkowników w czasie
- Wady:
  - Nie uwzględniamy zainteresowań użytkowników (każdemu rekomendujemy te same treści)
  - Zagładzamy większość artykułów, prezentując tylko te najbardziej interesujące dla ogółu (szczególnie dotkliwe w przypadku dużej puli różnorodnych artykułów)

Bandyci w segmentacji

- Dzielimy użytkowników na segmenty zgodnie z ich zainteresowaniami
- Dane o payoucie każdego materiału przechowujemy w rozbiciu względem segmentu
- Każde przychodzące zapytanie o rekomendację musi zawierać ID użytkownika
- W ramach pojedynczego segmentu wykorzystujemy algorytm bandyty (najlepiej z wygasającą wiedzą)

Bandyci w segmentacji

- Zalety:
  - Uwzględniamy zmienność zainteresowań użytkowników w czasie
  - W ograniczonym stopniu uwzględniamy różnorodne zainteresowania użytkowników
  - Zwiększamy różnorodność rekomendacji, w większym stopniu wykorzystujemy niszowe treści
  - Stosunkowo niewielki stopień skomplikowania z punktu widzenia aplikacji frontendowej (choć wymaga przechowywania stanu w postaci ID użytkownika)
- Wady:
  - Istotnie zwiększyliśmy stopień skomplikowania systemu (zarówno pod kątem przechowywania danych o payoucie jak i całego podsystemu obliczania i serwowania danych o segmentach)
  - Nadal nie uwzględniamy preferencji pojedynczych użytkowników

Metody oparte o embeddingi

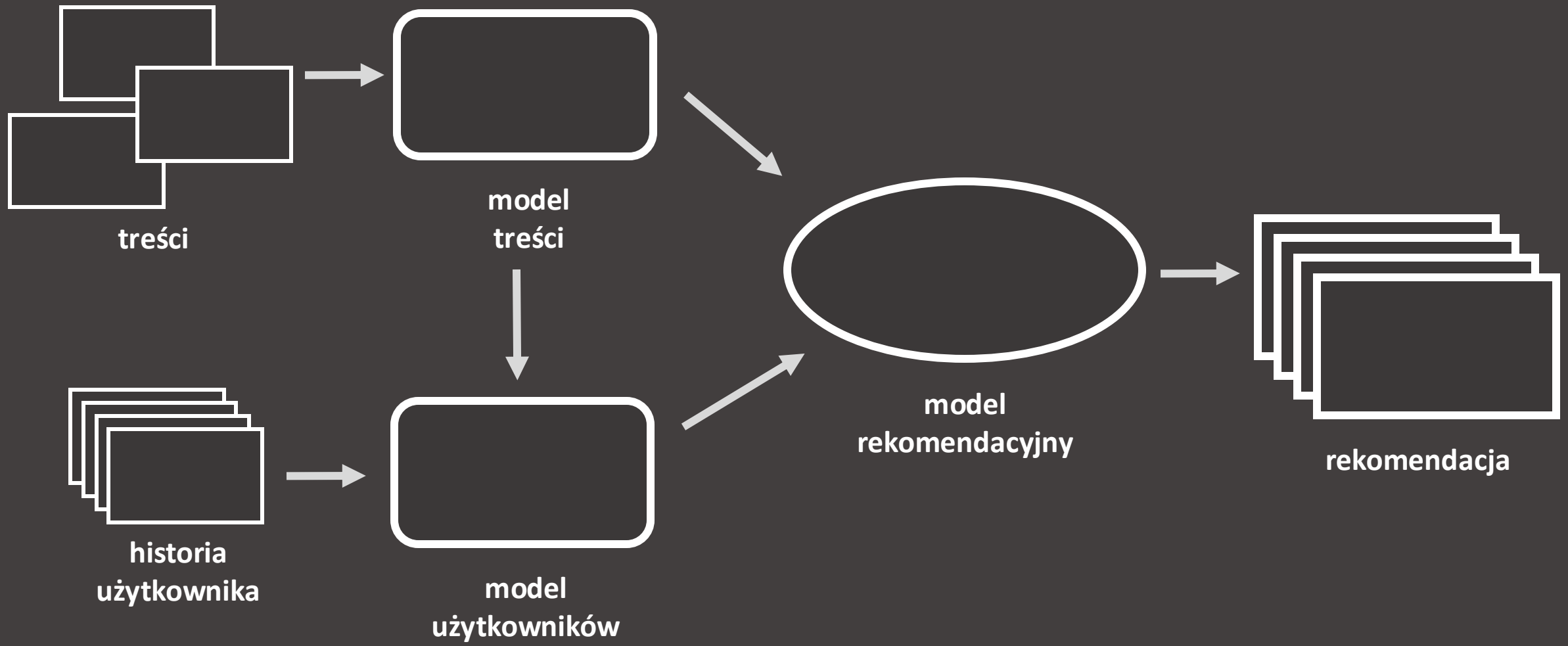
Problem

- Gusta i preferencje użytkowników zmieniają się wolno
- Nowe treści napływają cały czas
- Potrzebujemy podejścia, które może na bieżąco obliczać embeddingi nowych bytów (użytkowników i treści)

Antyprzykład – collaborative filtering

- CF w czasie treningu oblicza embeddingi treści i użytkowników jednocześnie
- Dodanie nowych użytkowników lub treści wymaga ponownego treningu modelu

Dekompozycja modelu



Model dla treści

- Dedykowany, gotowy model obliczający embeddingi dla danego typu treści
  - w przypadku tekstów – np. LLM
- Umożliwia szybkie obliczenie embeddingu
- Wynik warto zapisać w cache/bazie danych

Model dla użytkowników

- Wytrenowany przez nas
- Na wejściu przyjmuje historię aktywności użytkownika i embeddingi treści
- W najprostszej wersji – średnia z embeddingów treści z historii przeglądania
- Wynik warto zapisać w cache/bazie danych

Model rekomendacyjny

- Wytrenowany przez nas
- Na wejściu przyjmuje dwa embeddingi, treści i użytkownika
- Zwraca score – im wyższy, tym większe jest prawdopodobieństwo, że użytkownikowi dana treść się spodoba
- W najprostszej wersji – *cosine similarity*
- Poziom skomplikowania i czas odpowiedzi warto dostosować do możliwości systemu

Od tekstu do embeddingu

TF-IDF

$$w_{x,y} = tf_{x,y} \times \log\left(\frac{N}{df_x}\right)$$

TF-IDF

Term x within document y

$tf_{x,y}$ = frequency of x in y

df_x = number of documents containing x

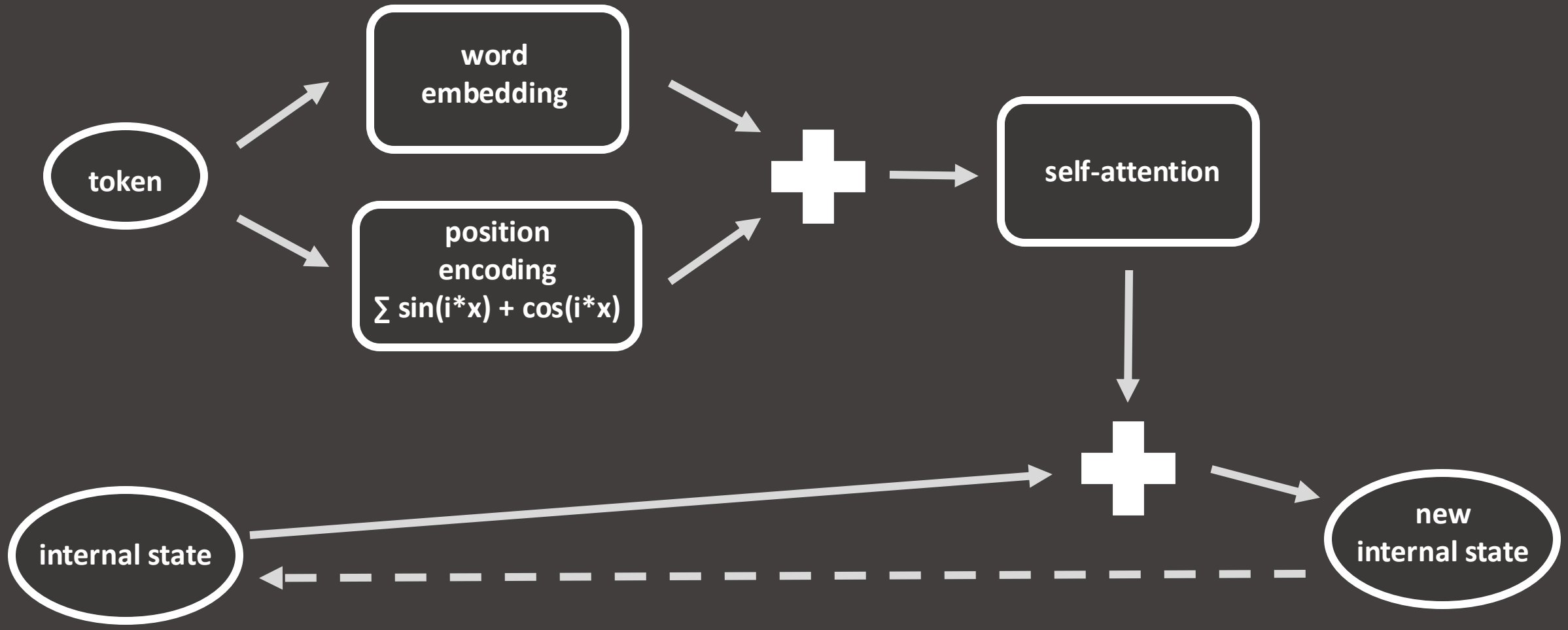
N = total number of documents

Źródło: https://ted-mei.medium.com/demystify-tf-idf-in-indexing-and-ranking-5c3ae88c3fa0

TF-IDF – implementacja

1. Preprocesing – sprowadzenie słów w całym korpusie do form podstawowych, usunięcie stop-words itp.
2. Utworzenie listy unikalnych tokenów w korpusie
3. *Bag of words* – reprezentacja każdego tekstu w korpusie jako wektora częstości tokenów
4. Obliczenie macierzy wartości *TF-IDF* na podstawie macierzy *bag of words*
5. Obliczenie macierzy podobieństwa (np. metryka cosinusowa)

LLM


token
word
embedding
position
encoding
∑ sin(i*x) + cos(i*x)
self-attention
internal state
new
internal state

LLM



LLM

- Umożliwia szybkie przekształcenie tekstu w embedding
- (zazwyczaj) wielojęzyczny
- Model wykorzystywany tylko do embeddingów nie musi być najnowszy i najbardziej zaawansowany z dostępnych
- Przykłady: GPT, BERT
- Przystępne i szczegółowe wytłumaczenie LLMów: https://youtu.be/zxQyTK8quyY

Automatyczne tagowanie

Kategoryzacja treści

- Zakładamy, że każdy tekst w naszym systemie może mieć przypisaną jedną lub kilka kategorii tematycznych
  - Kategorie mogą posiadać swoje podkategorie, np. Sport → Curling
- Ręczna kategoryzacja jest podatna na błędy, nawet jeśli dostarczymy zamknięty katalog kategorii
- Poprawna kategoryzacja umożliwi tworzenie sekcji tematycznych, np. Biznes, Sport, Technologie, Kultura







Proste metody nadzorowane

- Zakładamy, że mamy dostępny sensownych rozmiarów korpus tekstów z przypisanymi kategoriami
- Chcemy wytrenować prosty model przypisujący (albo chociaż proponujący) kategorie, do jakich należeć powinien nowopowstały tekst

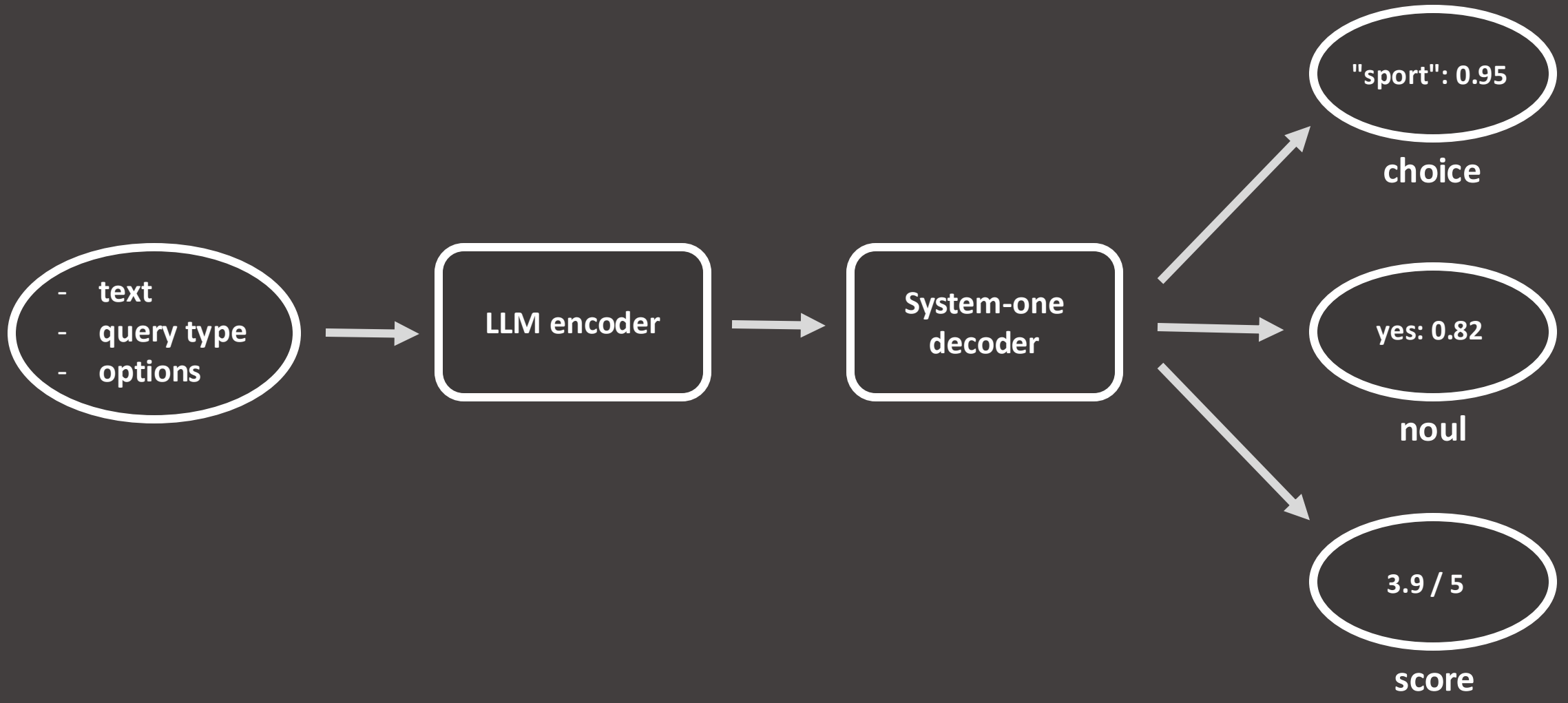
TF-IDF

- Budujemy prosty model predykcyjny
  - Np.: dla każdej kategorii wyznaczamy słowa, które w tekstach należących do tej kategorii miały najwyższy score *TD-IDF*
- Dla każdej nowej treści obliczamy wektor wartości *TF-IDF*
- Na podstawie słów o najwyższym *score* zgadujemy kategorie, do których należy nowy tekst

Jev i inne "system-one models"

- Modele pokrewne istniejącym LLM
- Używają encodera z LLM
- Wykonują jedną iterację dekodera zamiast podejscia rekurencyjnego
  - dzięki czemu mogą być znacznie tańsze
- Na wyjściu, zamiast tekstu odpowiedzi, zwracają jedną ze zdefiniowanych na wejściu opcji
- Pomysł nie jest zupełnie nowy, popularność zyskał we wrześniu 2026

Jev i inne "system-one models"



Jev w kategoryzacji treści

- W zapytaniu typu "choice" umieszczamy tekst oraz listę istniejących kategorii
- Model zwraca najbardziej prawdopodobną kategorię

Podsumowanie

- Jak uwzględnić w rekomendacji preferencje użytkowników?
- Rozdzielenie modelu na części
- Embeddingi użytkowników
- Embeddingi elementów
- Algorytmy tekstowe
- Kategoryzacja treści